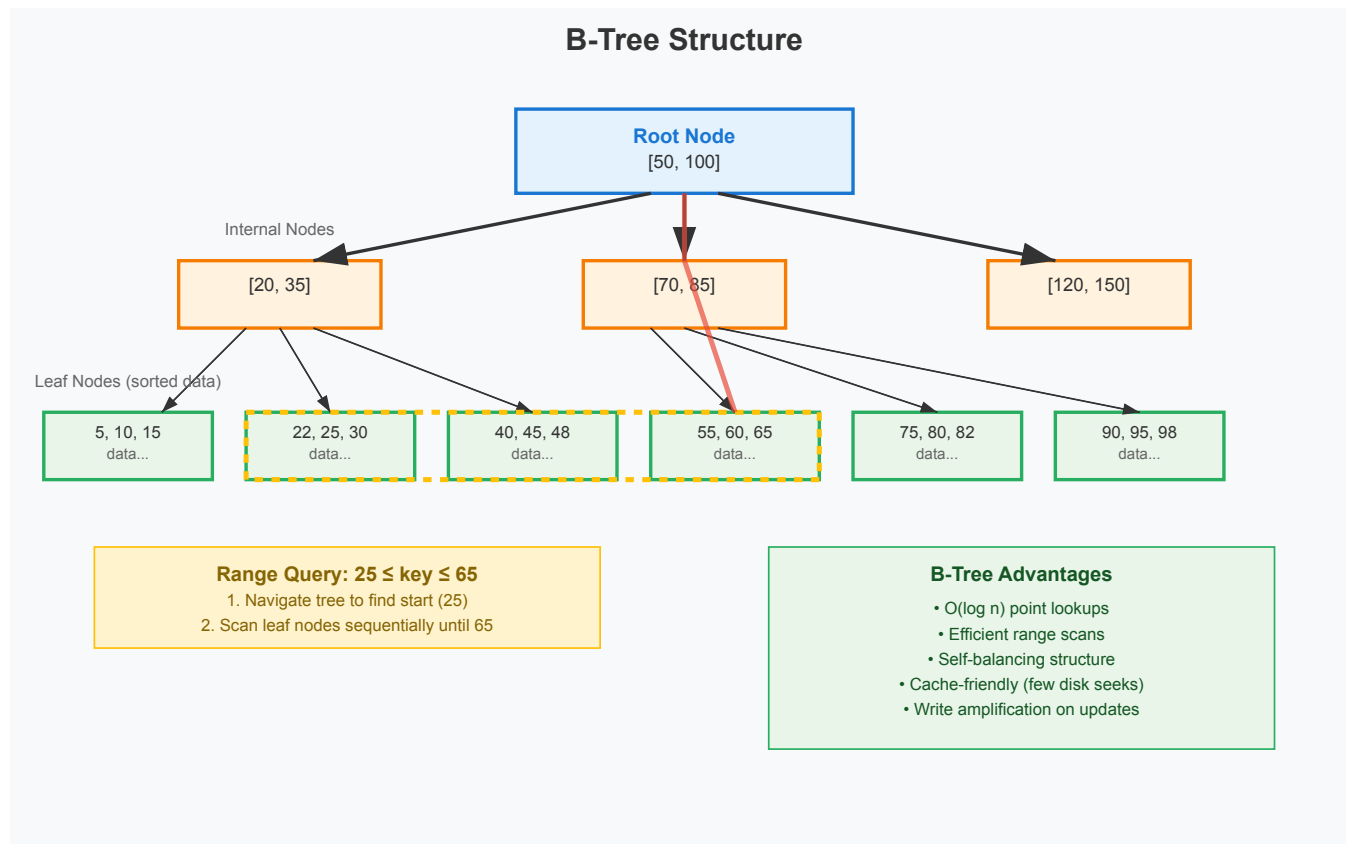


Understand Its Structure, Operations, and Use-Cases Now!

 systemdesignschool.io/fundamentals/b-tree



B-tree

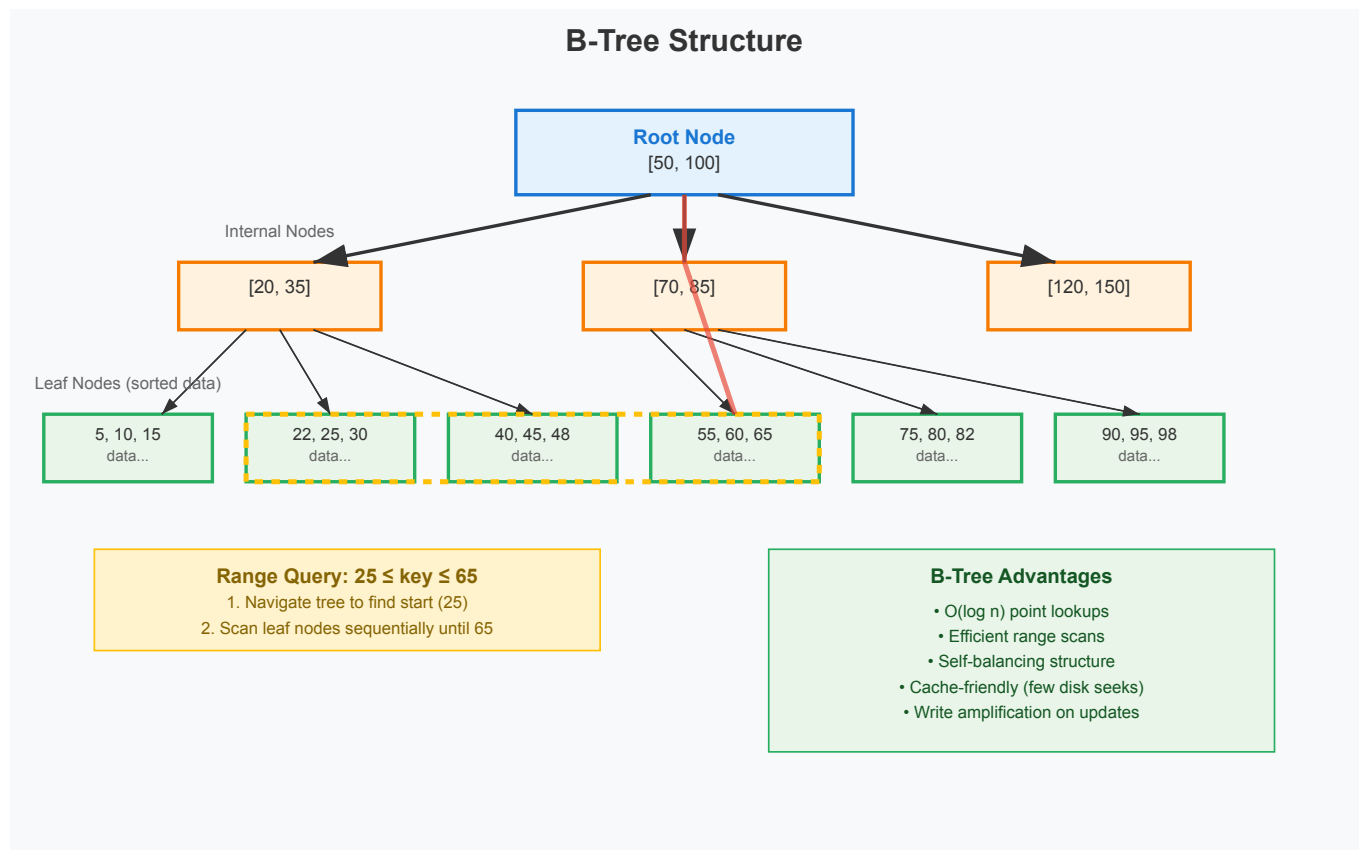
B-tree: Disk-Optimized Tree Structure

Binary search trees work well in memory but fail catastrophically on disk. Each node lookup requires a separate disk read. Searching through a million-record database means potentially thousands of disk operations, each taking 5-10 milliseconds. What should be a microsecond operation becomes seconds of waiting.

B-trees solve the disk access problem by packing hundreds of keys into each node. Instead of storing one key per node like binary trees, B-tree nodes contain 50-500 keys. A single disk read fetches an entire node, making each I/O operation maximally useful.

This architectural change transforms database performance. MySQL's InnoDB engine uses B+ trees (a B-tree variant) as its primary storage structure. PostgreSQL similarly relies on B-tree indexes. Without B-trees, relational databases would be unusably slow on traditional storage.

B-trees maintain three critical properties: they stay **balanced** (all leaf nodes at the same depth), **sorted** (keys within nodes are ordered), and **wide** (nodes contain many keys). These properties ensure consistent performance regardless of data insertion patterns.



Node Structure and Constraints

B-trees enforce strict structural rules that enable their disk optimization. Each **order m** B-tree has nodes containing between $\lceil m/2 \rceil - 1$ and $m - 1$ keys, except the root which can have as few as one key. This constraint ensures nodes stay between half-full and completely full, preventing degenerate cases that would hurt performance.

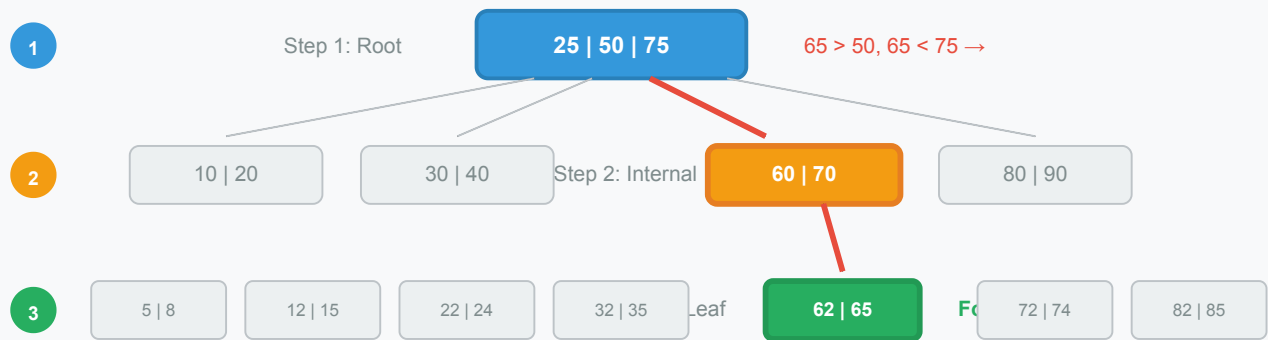
The **root node** sits at the top and contains up to $m - 1$ keys with m child pointers. **Internal nodes** contain keys plus pointers to child nodes. Keys within each node are sorted, allowing binary search within the node. **Leaf nodes** contain the actual data records along with their keys, and crucially, all leaves exist at the same tree depth.

For database storage, typical B-tree orders range from 50 to 500, meaning each node contains dozens to hundreds of keys. This matches disk page sizes (4KB-16KB), maximizing the data retrieved per disk read.

Search Operations

Finding a record in a B-tree requires walking from root to leaf, using binary search within each node. For a tree with one million records and order 100, this means just 3-4 disk reads maximum. Each node access narrows the search space dramatically.

B-tree Search: Finding Key 65



Search Performance

- 3 disk reads to find any record (root → internal → leaf)
- Each node contains 50-500 keys, maximizing data per disk access



The search algorithm compares the target key against keys in the current node. If found, the search succeeds. Otherwise, the algorithm follows the appropriate child pointer based on where the target key would fall in the sorted order.

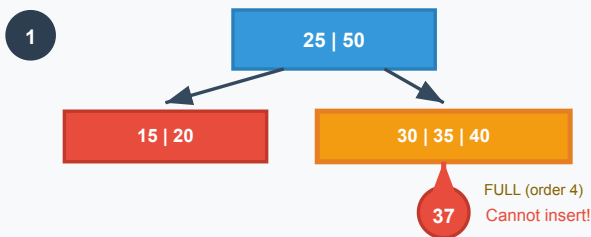
Range queries become especially efficient. After locating the starting key, the database can scan leaf nodes sequentially since they're sorted and often linked. This enables fast `SELECT * FROM users WHERE age BETWEEN 25 AND 35` queries without additional tree traversals.

Insertion and Node Splitting

B-tree insertions always happen at leaf nodes to maintain sorted order and consistent depth. When inserting into a non-full leaf, the operation simply places the new key in sorted position. However, when a leaf node becomes full (contains $m-1$ keys), **node splitting** occurs.

B-tree Insertion & Node Splitting Process

Step 1: Insert 37 into full leaf node



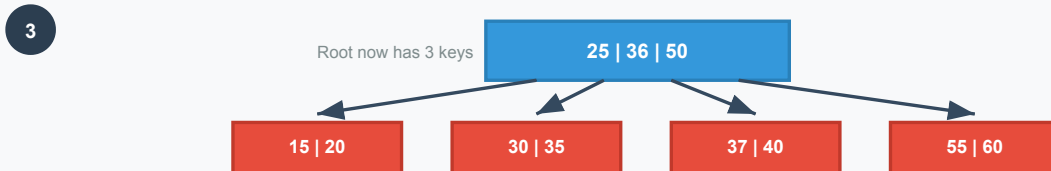
Step 2: Split leaf node



Performance Impact

- Splits are rare: only when nodes full
- Most insertions: $O(\log n) + 1$ write
- Split insertions: $O(\log n) + \text{up to } \log(n)$ writes (cascade up tree)
- Keeps nodes 50-100% full

Step 3: Final tree after split propagation



Node Splitting Algorithm

When inserting into a full leaf node ($m-1$ keys):

1. Insert new key into sorted position (temporarily creating m keys)
2. Split node: left gets $\lceil m/2 \rceil$ keys, right gets $\lfloor m/2 \rfloor$ keys
3. Promote middle key to parent node
4. If parent is full, repeat splitting process recursively
5. May create new root if splitting reaches the top



During splits, the node divides into two nodes, each containing $\lceil m/2 \rceil - 1$ keys. The middle key promotes to the parent node. If the parent is also full, splitting cascades upward, potentially creating a new root and increasing tree height by one level.

This splitting mechanism maintains the half-full guarantee while keeping the tree balanced. Modern databases optimize splits by maintaining pages that are $2/3$ full rather than completely full, reducing split frequency during bulk insertions.

Production Database Usage

MySQL's InnoDB engine implements B+ trees (a B-tree variant where data resides only in leaves) for both primary and secondary indexes. The clustered index organizes table data in primary key order, while secondary indexes contain pointers back to the clustered index.

PostgreSQL uses B-trees for most index types, with typical page sizes of 8KB containing 100-200 index entries. PostgreSQL's implementation includes optimizations like right-link pointers between sibling pages to improve concurrent access patterns.

File systems also rely on B-trees. **ext4** uses extent trees (a B-tree variant) to track file block allocations efficiently. **NTFS** organizes directory entries and file allocation tables using B+ tree structures.

B-tree Search Operation

```
def __init__(self, keys=None, children=None, is_leaf=False):
    self.keys = keys or []
    self.children = children or []
    self.is_leaf = is_leaf
    def btree_search(node, target_key, path=None):
        """Search for a key in B-tree, tracking the path taken"""
        path.append(f"Visiting node with keys: {node.keys}")
        # Binary search within the node
        for i, key in enumerate(node.keys):
            if target_key == key:
                path.append(f"Found key {target_key} at position {i}")
                return target_key, path
            elif target_key < key:
                if node.is_leaf:
                    path.append(f"Key {target_key} not found - reached leaf")
                    return None, path
                path.append(f"Key {target_key} < {key}, following left child")
                return btree_search(node.children[i], target_key, path)
        # Key is greater than all keys in node
        path.append(f"Key {target_key} not found - reached leaf")
        return None, path
        path.append(f"Key {target_key} > all keys, following rightmost child")
        return btree_search(node.children[-1], target_key, path)
    # Build a sample B-tree (order 4)
```